



Programming Foundations Level 1



Programming Foundations

Level 1

Operator

Operators in Computers: Math, Relational, and Logic

Operators are the **symbols** that tell the computer **what action to perform** on data. In this lesson you'll learn the three core families you'll use everywhere:

- **Mathematical operators** (e.g., +, -, *, /, Mod, ^)
- **Relational operators** (e.g., =, <, >, <=, >=, <>)
- **Logical operators** (e.g., AND, OR, NOT)

You'll also see **truth tables**, how computers view **True = 1** and **False = 0**, and realistic decision examples.

Operator Types

1

Mathematical

+ - / * modulo ^

2

Relational

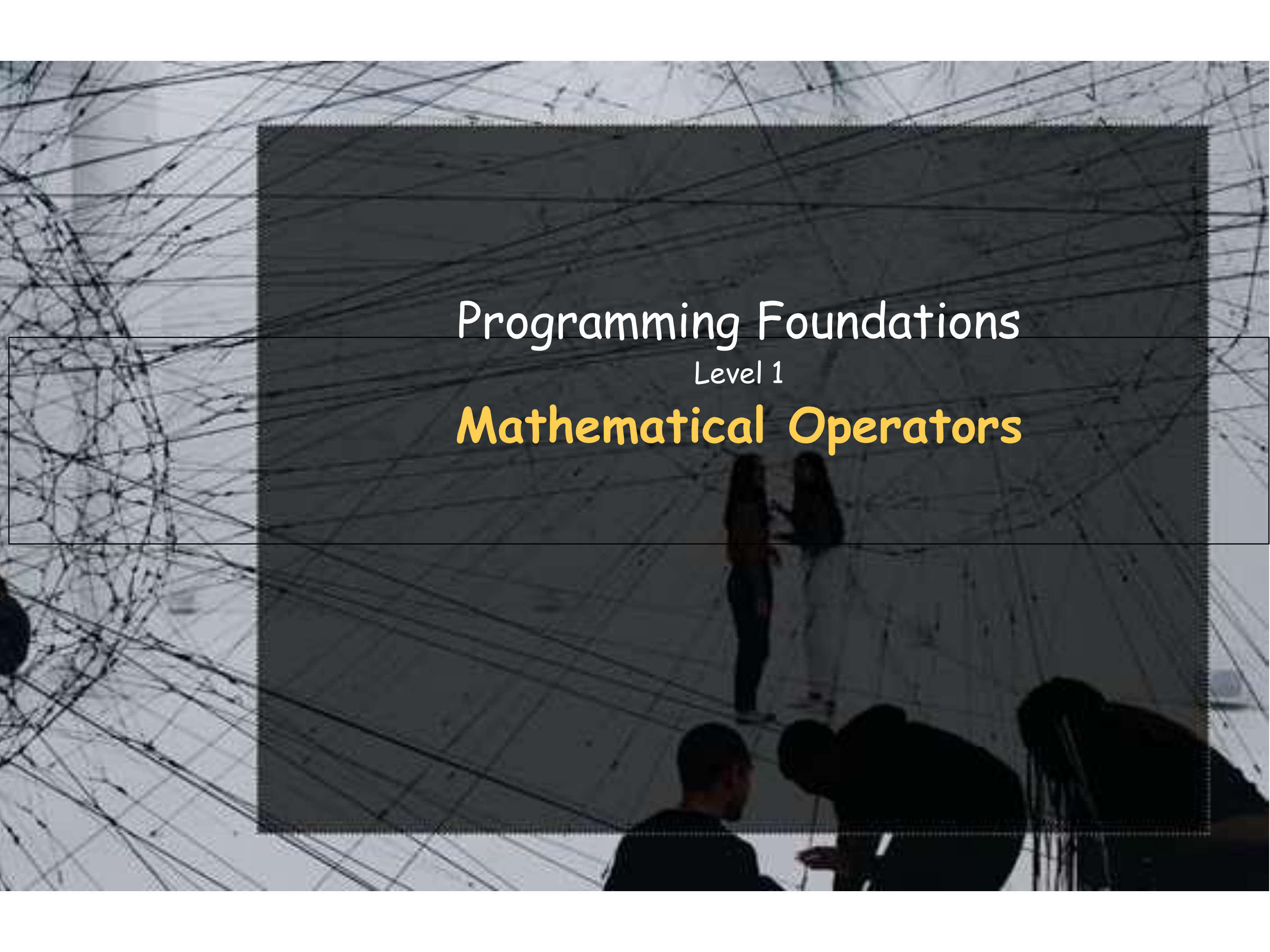
>, <, =, >=, <=, <>

3

Logical

And, Or, NOT

- Mathematical: arithmetic on numbers
- Relational: compare two values → returns True/False
- Logical: combine/negate conditions (True/False)



Programming Foundations

Level 1

Mathematical Operators

Mathematical Operators

Operator	Computer Symbol	Example	
		Operation	Results
Addition	+	3.0 + 4.2	7.2
Subtraction	-	8.5 - 4.0	4.5
Multiplication	*	5 * 5	25
Integer Division	/	10 / 5	2
Modulo Division	Mod	9 Mod 4	1
Power	^	3^2	9



Programming Foundations

Level 1

Relational Operators

Relational Operators

Operator	Computer Symbol	Example	
		Operation	Results
Equal to	=	5 = 7	False
Less than	<	5 < 7	True
Grater than	>	5 > 7	False
Less than or equal to	<=	5 <= 7	True
Grater than or equal to	>=	5 >= 7	False
Not equal to	<>	5 <> 7	True

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning forward as if working on a table or screen.

Programming Foundations

Level 1

Logical Operators

The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, including two standing figures in the center and a group of people in the foreground, some of whom appear to be looking at a screen or display. The overall aesthetic is technical and interconnected.

Programming Foundations

Level 1

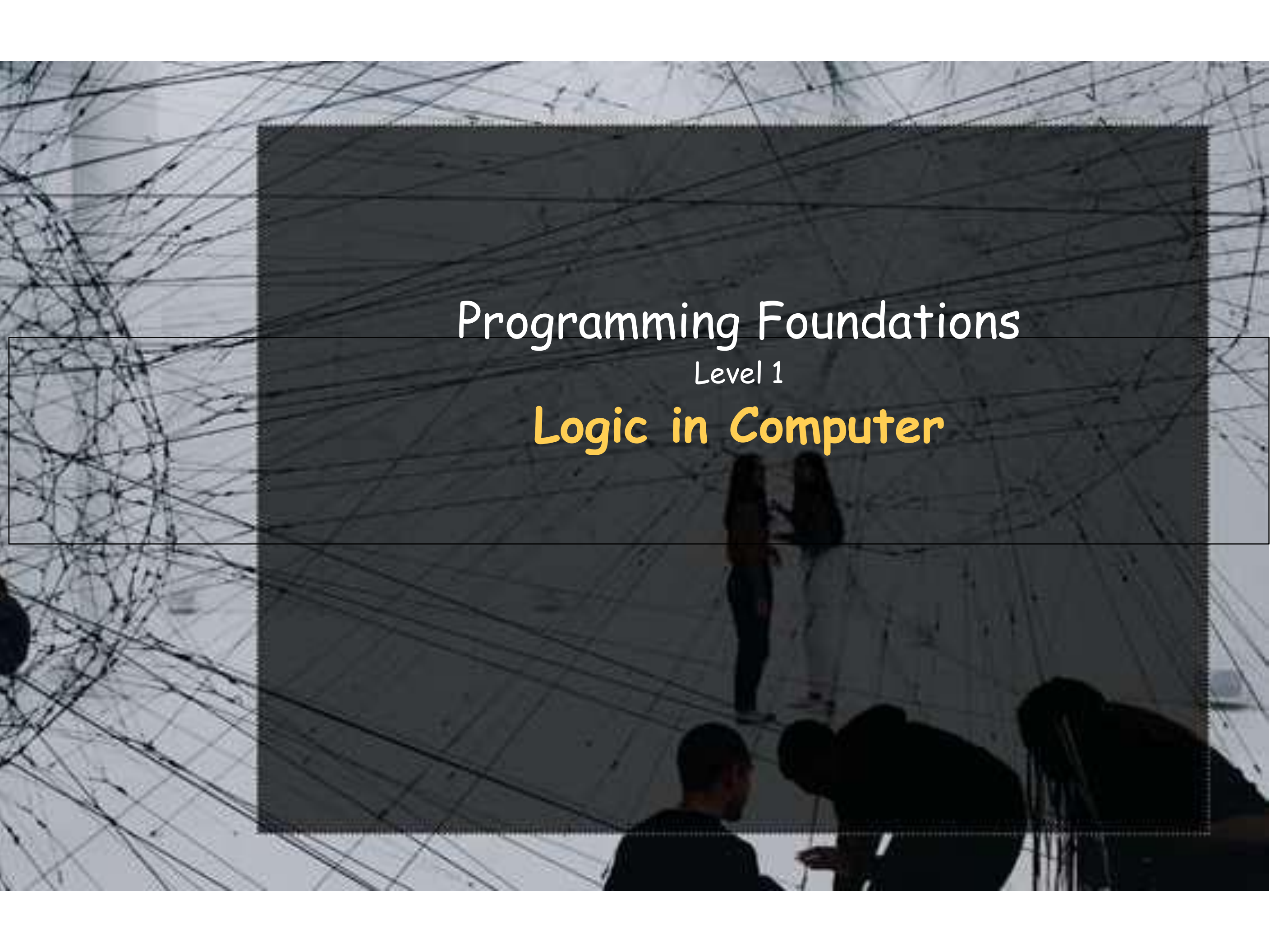
Logic in Marriage

Logic in Marriage Life



Husband	Wife	Result
True	False	True
True	True	True
False	True	True
False	False	True

Your wife is always right no matter what, and you are always wrong :-)

The background of the slide is a dark, abstract image. It features a dense, intricate network of thin, light-colored lines that resemble a complex web or a neural network. In the lower portion of the image, there are dark silhouettes of several people. Two people are standing in the center, facing each other, while others are visible in the foreground, some appearing to be looking at a screen or device. The overall aesthetic is technical and modern.

Programming Foundations

Level 1

Logic in Computer

The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a group, which adds a human element to the technical-looking background.

Programming Foundations

Level 1

Logic Operators



Programming Foundations

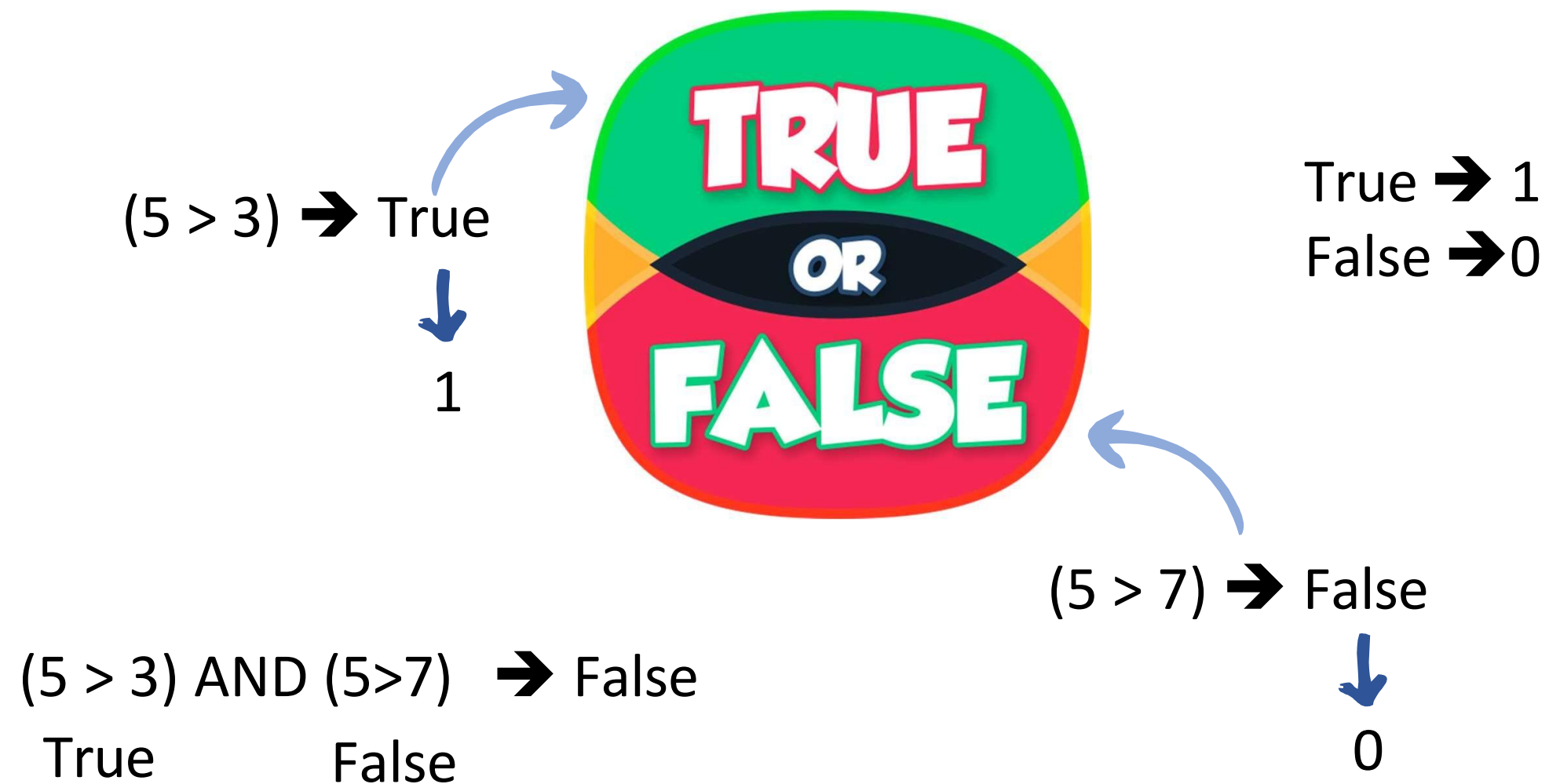
Level 1

"AND" Operators

Relational Operators

Operator	Computer Symbol	Example	
		Operation	Results
And	AND	True AND True	True
Or	OR	True OR False	True
Not	NOT	NOT True	False

Logical Operators



Logical Operators

The slides illustrate logic with everyday statements ("logic in marriage/life") and then map that to computer-style logic with **AND / OR / NOT**. This helps you see how **conditions** become **code decisions**.

Practical AND example (hiring rule):

"You must meet **both** conditions to be hired:

- Age $\geq$ 21 **AND** Has a driver license.

Computer form: (Age $\geq$ 21) AND (HasDriverLicense = True)

A single **False** in AND $\rightarrow$ result **False** (not hired).

Practical OR example (hiring rule):

"You must meet **at least one**:

- Age $\geq$ 21 **OR** Has a driver license."

Computer form: (Age $\geq$ 21) OR (HasDriverLicense = True)

Any **True** in OR $\rightarrow$ result **True** (hired).

Logical “AND” Operator:

Condition A	Condition B	A AND B
Age	Has Driver License	
25 >= 21 → True	Yes → True	True
25 >= 21 → True	No → False	False
19 >= 21 → False	Yes → True	False
19 >= 21 → False	No → False	False

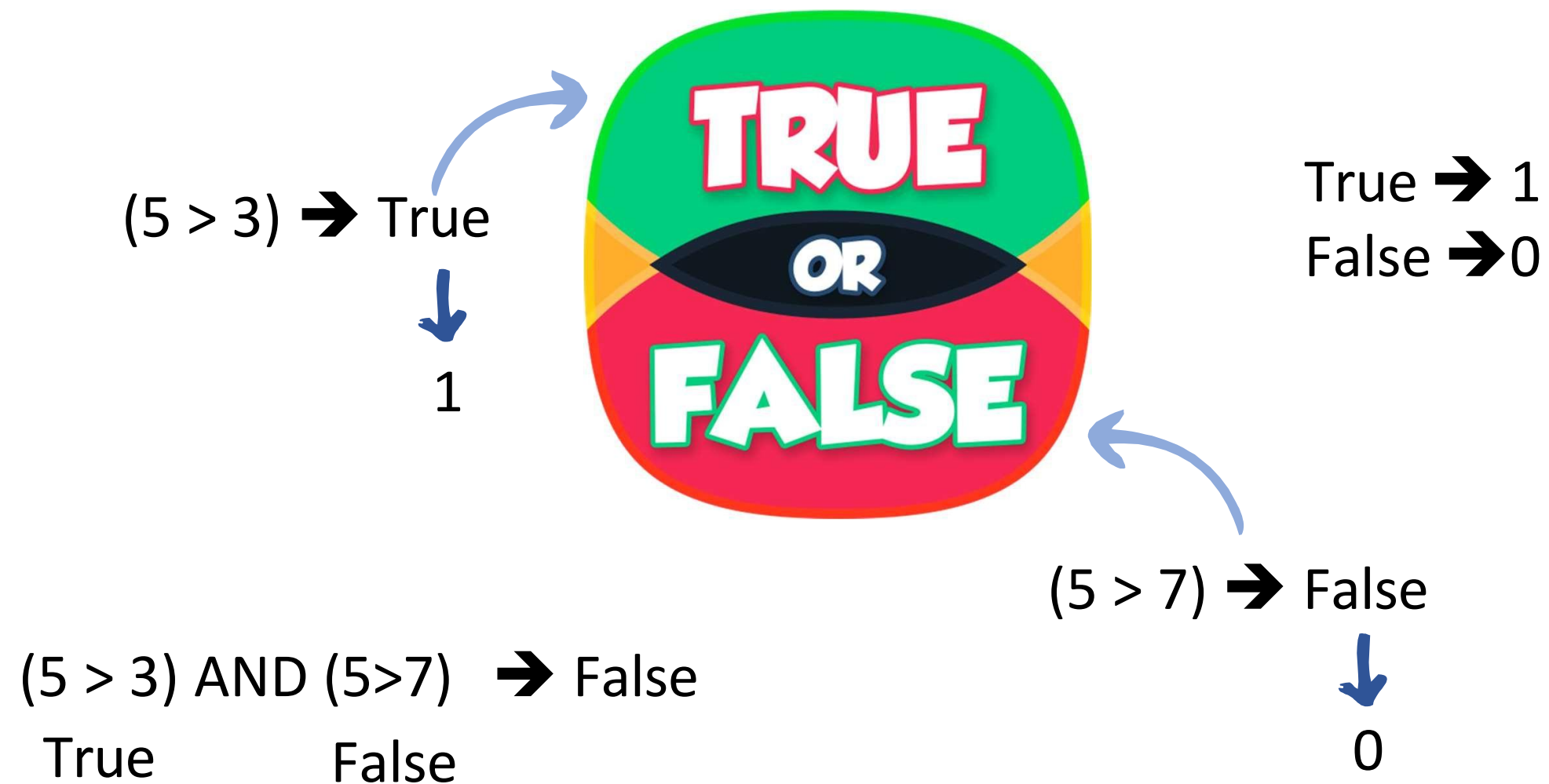
AND

Any "False" in AND
the result is False

A	B	A AND B
True	True	True
True	False	False
False	True	False
False	False	False

A	B	A AND B
1	1	1
1	0	0
0	1	0
0	0	0

Logical Operators



Practical Example to learn "AND" operator:

IF you wan to get hired you have to have all the following conditions:

1. Your Age must be ≥ 21 years
2. You should have a driver license.

This in computer means:

(Age $\geq$ 21 **AND** HasDriverLicesne=True) Then will be hired



Programming Foundations

Level 1

“OR” Operators

Logical “OR” Operator:

Condition A	Condition B	A OR B
Age	Has Driver License	
25 >= 21 → True	Yes → True	True
25 >= 21 → True	No → False	True
19 >= 21 → False	Yes → True	True
19 >= 21 → False	No → False	False

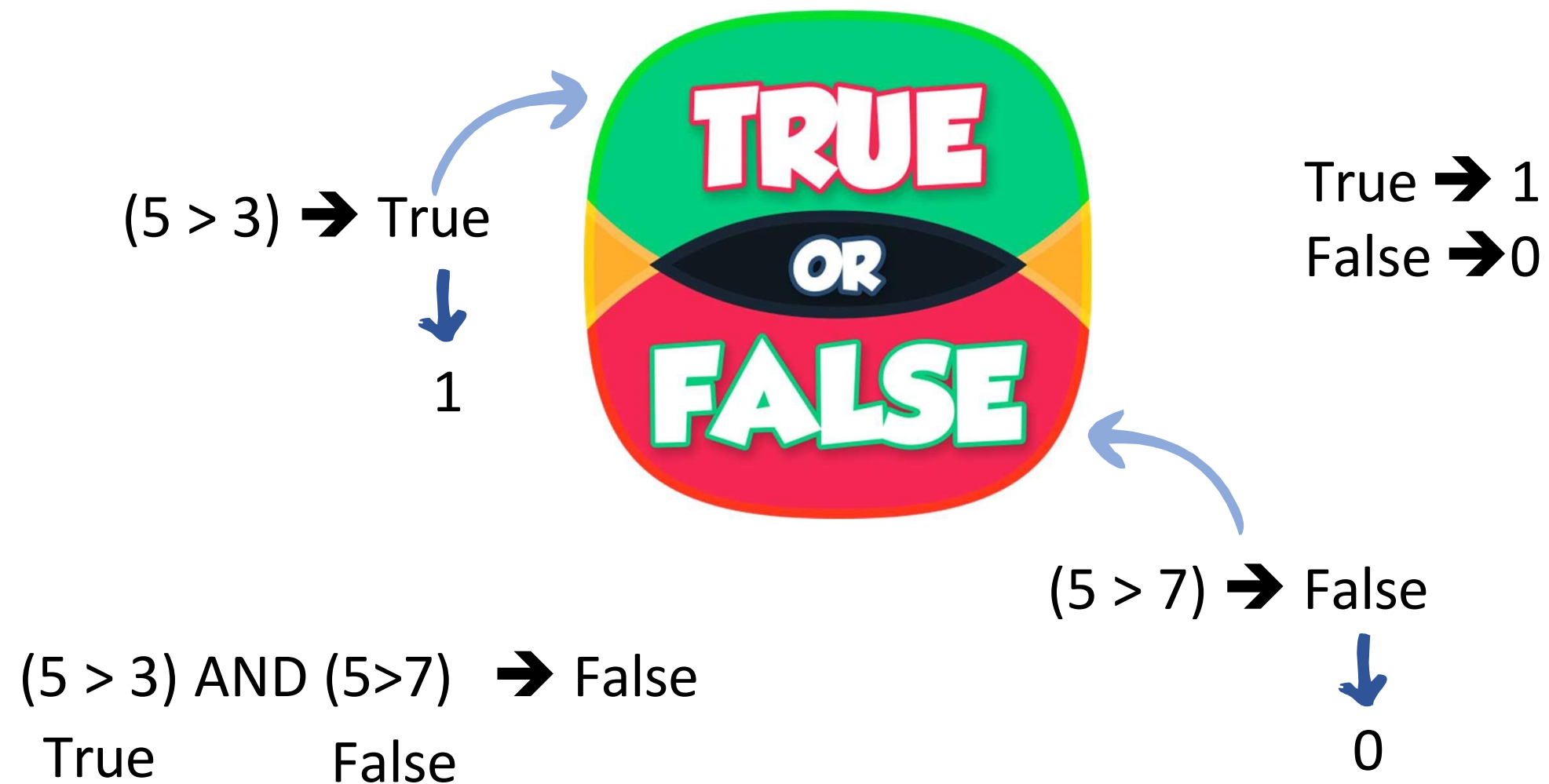
OR

Any "True" in OR
the result is True

A	B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

A	B	A OR B
1	1	1
1	0	1
0	1	1
0	0	0

Logical Operators



Practical Example to learn "OR" operator:

IF you want to get hired you have to have at least One the following conditions:

1. Your Age must be ≥ 21 years
2. You should have a driver license.

This in computer means:

(Age $\geq$ 21 **OR** HasDriverLicense=True) Then will be hired



Programming Foundations

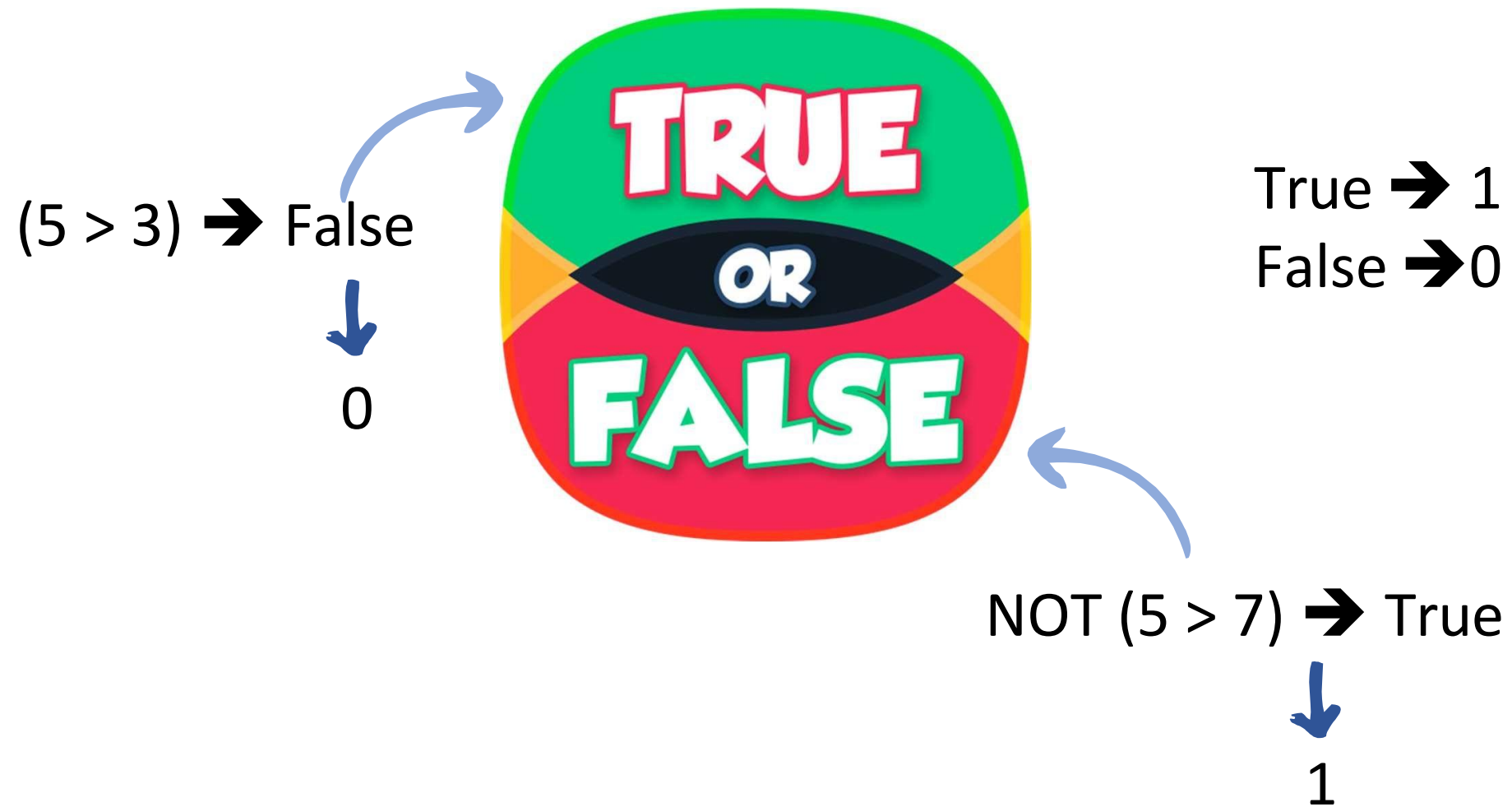
Level 1

“NOT” Operators

Logical “NOT” Operator:

Condition A	NOT A
True	False
False	True

Logical Operators



Practical Example to learn "NOT" operator:

I want to hire only Males , so you can say in computer:

If Male then hire

or you can use Not: If not (Female) then hire. because not female means male

Interconnection

- ◆ **Math operators** compute values; **Relational** compare values; **Logical** combine decisions.
- ◆ Computers evaluate conditions as **True/False** (i.e., 1/0) and use **AND/OR/NOT** to decide flow.
- ◆ Real policies (like hiring rules) translate directly into **logical expressions** you can code.

By mastering these, you'll express real-world rules precisely and build reliable decisions in programs. 🚀



Activity



Instructions:

Decide True/False or compute the result.

1 12 >= 12

2 12 > 7

3 8 < 6

4 8 = 8

5 12 <= 12

6 7 = 5

7 (7 = 7) AND (7 > 5)

8 (7 = 7) AND (7 < 5)

9 (7 = 7) OR (7 < 5)

10 (7 < 7) OR (7 > 5)

1 1 NOT (7 = 7) AND (7 > 5)

1 2 (7 = 7) AND NOT (7 < 5)

1 3 NOT (12 >= 12)

1 4 NOT (12 < 7)

1 5 NOT (8 < 6)

1 6 NOT (8 = 8)

1 7 NOT (12 <= 12)

1 8 NOT (7 = 5)



Activity Solution



Solution:

✓ Solved Table

#	Expression	Evaluation Steps	Result
1	$12 \geq 12$	12 is equal to 12	True
2	$12 > 7$	12 is greater than 7	True
3	$8 < 6$	8 is not less than 6	False
4	$8 = 8$	Both values are equal	True
5	$12 \leq 12$	12 is equal, so $\leq$ holds	True
6	$7 = 5$	7 is not equal to 5	False
7	$(7 = 7) \text{ AND } (7 > 5)$	True AND True	True
8	$(7 = 7) \text{ AND } (7 < 5)$	True AND False	False
9	$(7 = 7) \text{ OR } (7 < 5)$	True OR False	True
10	$(7 < 7) \text{ OR } (7 > 5)$	False OR True	True
1 1	$\text{NOT } (7 = 7) \text{ AND } (7 > 5)$	(NOT True) AND True $\rightarrow$ False AND True	False
1 2	$(7 = 7) \text{ AND NOT } (7 < 5)$	True AND (NOT False $\rightarrow$ True)	True
1 3	$\text{NOT } (12 \geq 12)$	NOT True	False
1 4	$\text{NOT } (12 < 7)$	NOT False	True
1 5	$\text{NOT } (8 < 6)$	NOT False	True
1 6	$\text{NOT } (8 = 8)$	NOT True	False
1 7	$\text{NOT } (12 \leq 12)$	NOT True	False
1 8	$\text{NOT } (7 = 5)$	NOT False	True

Conclusion

- ✓ **Math operators** compute values; **relational** compare; **logical** combine decisions.
- ✓ Computers reduce conditions to **True/False (1/0)** and use **AND/OR/NOT** to decide program flow.
- ✓ Real policies map cleanly to logical expressions — a foundation you'll reuse in **every programming language**.

Thank you 😊